2024

Cloud Blog Deployment with WordPress

DOCUMENTATION OF SETUP PROCESS

ZANE GROSE

MEDIA DESIGN SCHOOL | ICC104

Contents

Introduction 2
Cloud Provider selection 2
Project setup and process documentation 2
Account registration and VM setup 2
Deploying WordPress 5
Cost and resource management 6
Reflection 7
Comparison 9
Conclusion 9

Introduction

In this report, I will be detailing the setup, decisions, and processes I have made to deploy a WordPress blog using the LAMP (Linux, Apache, MySQL, and PHP) technology stack utilizing a cloud provider. The cloud allows developers to host websites by utilizing computing power such as VM's and containers, store data that can hold website and database information, and network infrastructure where load balancers, firewalls, and VPNs can be utilized to increase security and accessibility. The cloud provides cost-effectiveness through a pay-as-you-go or usage-cost models, reliability due to the scale of cloud providers, and scalability thanks to the diverse selection of data centres worldwide.

Cloud Provider selection

The cloud provider I selected was Microsoft Azure, due to the user interface being easier to use compared to AWS or Google Cloud in my opinion, having an email with Microsoft allows collaboration with other well-known Microsoft services, the availability of quality documentation, and the tutorial availability for setting up a WordPress blog using LAMP stack, which I accessed from Microsoft Learn (Shim, et al., 2024). Alongside the WordPress Blog LAMP stack walkthrough, I also utilized the Azure CloudShell documentation (sdwheeler, 2024) which helped me in setting up the CloudShell to further setup the cloud-hosted WordPress blog. I am quite familiar working with PHP, MySQL, and Linux in a CLI, which is another reason why I decided to choose Azure. The easy setup and usage of the CloudShell feature and the similarities with all the technologies I have used was a decision maker.

Project setup and process documentation

Account registration and VM setup

Signing in to my MDS email, I registered an Azure account, through which I entered my address and card details. Microsoft then prompted me to secure my account using their authenticator app, I downloaded the app on my phone and verified my identity. This allowed me to sign up to Microsoft Azure quickly, while also securing my account.

Setting up the VM was done utilizing the CloudShell (CLI) feature of Azure. I initially had to register CloudShell to my subscription package, by following these steps:

1. Azure Dashboard ->
2. Subscriptions ->
3. Azure subscription 1 ->
4. Resource providers ->

5. (select) Microsoft.CloudShell ->
6. Register.

Steps 4-6 are shown in *Figure 1*, this dashboard is accessed by following steps 1-3.

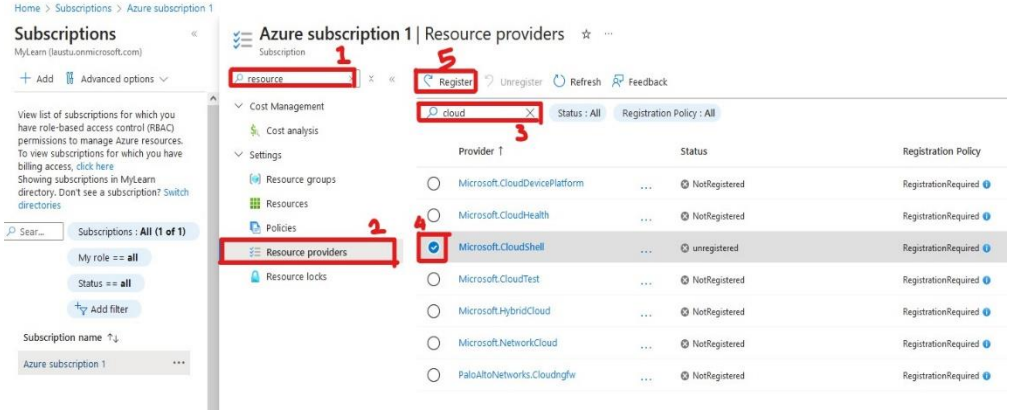


Figure 1: Steps taken to enable CloudShell for Azure subscription, labelled in order from 1-5

Before creating the VM, the resources needed to be stored, this was done by creating a resource group, shown in *Figure 2* by using the CLI.

```
zane [ ~ ]$ az group create --name WordPressBlog --location southeastasia
{
  "id": "/subscriptions/053c9d57-5aff-4fd1-ba84-9dcb1625d307/resourceGroups/WordPressBlog",
  "location": "southeastasia",
  "managedBy": null,
  "name": "WordPressBlog",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

Figure 2: Creating a resource-group to store resources

This enabled me to access the bash CLI to further create the VM hosted on the cloud. I entered the command in the CLI shown in *Figure 3* which created a cloud-hosted Linux VM. I deprecated the mac, private, and public IP addresses considering security.

```
zane [ ~ ]$ az vm create --resource-group WordPressBlog --name LinuxVM --image Ubuntu2204 --admin-username zadmin --generate-ssh-keys
/usr/lib64/az/lib/python3.9/site-packages/paramiko/pkey.py:100: CryptographyDeprecationWarning: TripleDES has been moved to cryptograph
  "cipher": algorithms.TripleDES,
/usr/lib64/az/lib/python3.9/site-packages/paramiko/transport.py:259: CryptographyDeprecationWarning: TripleDES has been moved to crypto
  "class": algorithms.TripleDES,
SSH key files '/home/zane/.ssh/id_rsa' and '/home/zane/.ssh/id_rsa.pub' have been generated under ~/.ssh to allow SSH access to the VM.
{
  "fqdns": "",
  "id": "/subscriptions/053c9d57-5aff-4fd1-ba84-9dcb1625d307/resourceGroups/WordPressBlog/providers/Microsoft.Compute/virtualMachines/L
  "location": "southeastasia",
  "macAddress": "[redacted]",
  "powerState": "VM running",
  "privateIpAddress": "[redacted]",
  "publicIpAddress": "[redacted]",
  "resourceGroup": "WordPressBlog",
  "zones": ""
}
```

Figure 3: CloudShell command to create a Linux VM, utilizing the previously created resource-group

After the VM was created, it was necessary to open port 80 (HTTP) for remote SSH access to the VM. This was done using the '*az vm open-port –port 80 –resource-group WordPressBlog –name LinuxVM*' command within the CLI. This enabled me to remotely access the VM through the CloudShell CLI, shown in *Figure 4*.

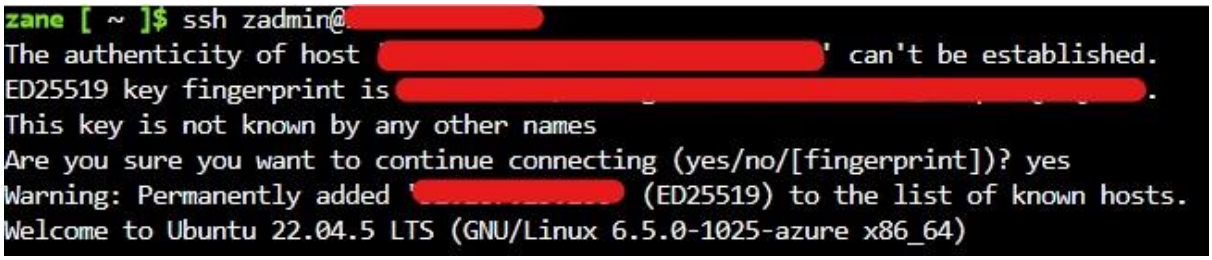


Figure 4: Remote access to the VM through the CloudShell CLI, with an initial security warning prompt

The initial warning given when trying to remotely connect is a security feature, ensuring that you're connecting to a secure IP, since I setup the VM, I am aware what I am connecting to, entering 'yes' to confirm.

The VM was then updated and LAMP stack installed, utilizing the command '*sudo apt update && sudo apt install lamp-server^*'. Ensuring the installation was successful, I checked Apache, MySQL, and PHP servers. Apache was checked by showing the version installed on the VM, shown in *Figure 5*. This was accompanied by entering the public IP, deprecated in *Figure 3*, using a web browser on my local machine shown in *Figure 6*.

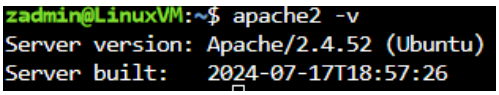


Figure 5: CLI command to check the Apache2 version installed on the VM

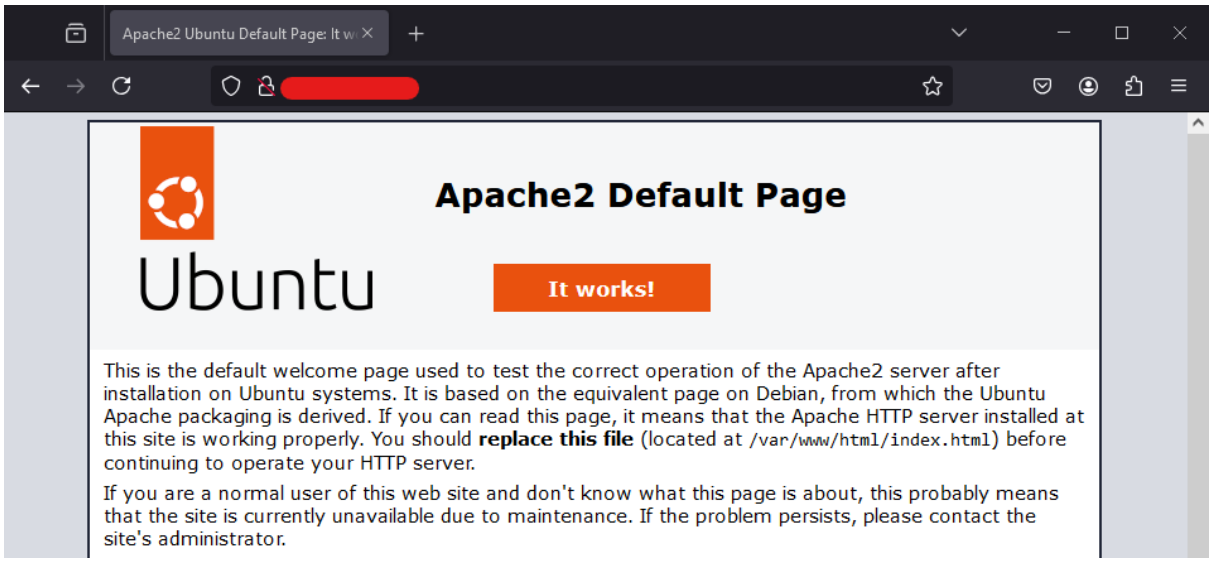


Figure 6: Accessing the public IP for the Apache2 server, ensuring its accessible via local machine

Next was to secure and verify the MySQL server.

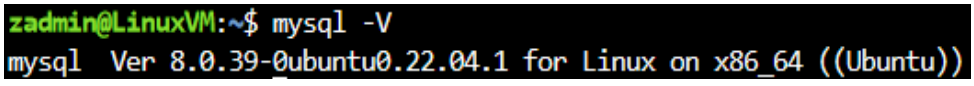


Figure 5: CLI command to check the MySQL version installed on the VM

Figure 7 shows that MySQL version 8.0.39 is successfully installed on the VM.

I then secured the MySQL server root account by changing the default blank password by accessing the MySQL terminal using '*sudo mysql -u root*' and entering the

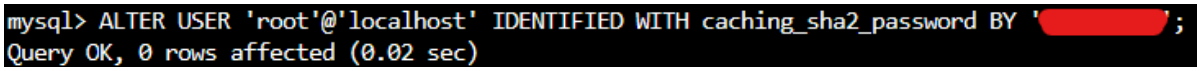


Figure 6: Command in MySQL terminal to change the root password

command

shown in *Figure 8*, using the sha2 caching plugin for password storage.

This new password was tested by entering the previous '*sudo mysql -u root*' command, with the an error response being generated. I then attempted to access the MySQL server using the following command this time; '*sudo mysql -u root -p*', where I was prompted to enter my password, and let into the server entering the previously set password in, shown in *Figure 9*.

```
zadmin@LinuxVM:~$ sudo mysql -u root
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: NO)
zadmin@LinuxVM:~$ sudo mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 8.0.39-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Figure 7: Testing root password access to MySQL server running on VM

Testing PHP was the last task, this was done and confirmed by using the '*php -v*' command shown in *Figure 10*. To further test PHP was enabled on the VM, I quickly created a PHP info page utilizing the command '*sudo sh -c 'echo "<?php phpinfo(); ?>" > /var/www/html/info.php'* '. This page was checked by going to my local device and entering '0.0.0.0/info.php' (where 0.0.0.0 is the public IP to my VM), to which *Figure 11* shows the results.

```
zadmin@LinuxVM:~$ php -v
PHP 8.1.2-1ubuntu2.19 (cli) (built: Sep 30 2024 16:25:25) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.1.2, Copyright (c) Zend Technologies
    with Zend OPcache v8.1.2-1ubuntu2.19, Copyright (c), by Zend Technologies
```

Figure 10: CLI Command to check the PHP version installed on the VM

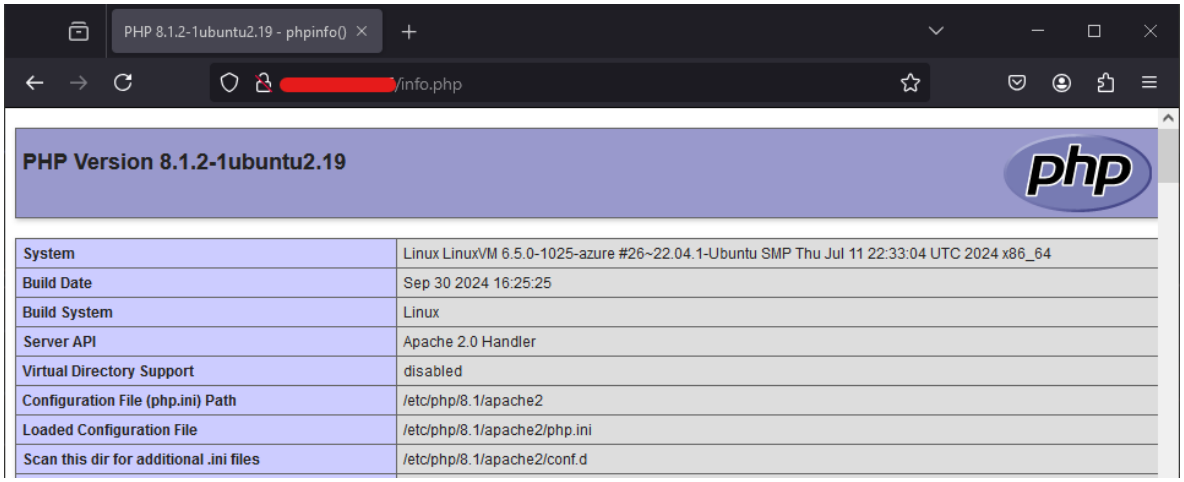


Figure 11: info.php page hosted on the VM IP

Deploying WordPress

First step in deploying WordPress to the cloud was to install it on the VM using the command '*sudo apt install wordpress*'.

```
 GNU nano 6.2                                    wordpress.sql
CREATE DATABASE wordpress;

CREATE USER 'wpadmin'@'localhost' IDENTIFIED BY '[redacted]';

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER
ON wordpress.*
TO 'wpadmin'@'localhost';
FLUSH PRIVILEGES;
```

Figure 8: .sql file with MySQL commands to create a database, user, and grant privileges

Creating a new .sql file in the VM directory and editing it using

Linux's Nano text editor, I entered the SQL commands to create a new database, user, and grant privileges, outlined in *Figure 12*. This .sql file was then used in conjunction with the command '*cat wordpress.sql | sudo mysql --defaults-extra-file=/etc/mysql/debian.cnf'* to create the database, user, and grant privileges. The .sql file was removed from the VM directory afterwards, in consideration of credentials being in plaintext.

Then I connected the MySQL database with the wordpress website using PHP, creating a config file for wordpress using the command '*sudo nano /etc/wordpress/config-localhost.php*'. This opened Nano text editor in the VM, to which I entered the database details in PHP code, shown in *Figure 13*.

```
 GNU nano 6.2                        /etc/wordpress/config-localhost.php
<?php
define('DB_NAME', 'wordpress');
define('DB_USER', 'wpadmin');
define('DB_PASSWORD', '[redacted]');
define('DB_HOST', 'localhost');
define('WP_CONTENT_DIR', '/usr/share/wordpress/wp-content');
?>
```

Figure 9:config-localhost.php file contents to connect MySQL database with the WordPress website, using PHP code

The final steps were to link the wordpress installation with the public access folder on the web server using the command '*sudo ln -s /usr/share/wordpress /var/www/html/wordpress*', allowing access to the wordpress blog through HTTP. Then finally moving the config file to be the default configuration file using this command '*sudo mv /etc/wordpress/config-localhost.php /etc/wordpress/config-default.php',* which configures wordpress to utilize the MySQL database created on the web server.

Cost and resource management

Azure provides free trial to resource groups and virtual machines without any additional costs for the first month of subscription, they also give a $330 credit to your azure subscription (for the first month). These features allow me to host the WordPress blog on the cloud at no costs for this project and even try features utilizing the credit given. Along with this, Azure provides MySQL databases features, but hosting a MySQL database on the VM supplied by Azure is at no additional cost. This reduced the cost of hosting the WordPress website.

To manage & monitor the cloud resources used for this project, there are many options in Azure. One of the common ones I used was cost management -> cost analysis. Shown in *Figure 14*,

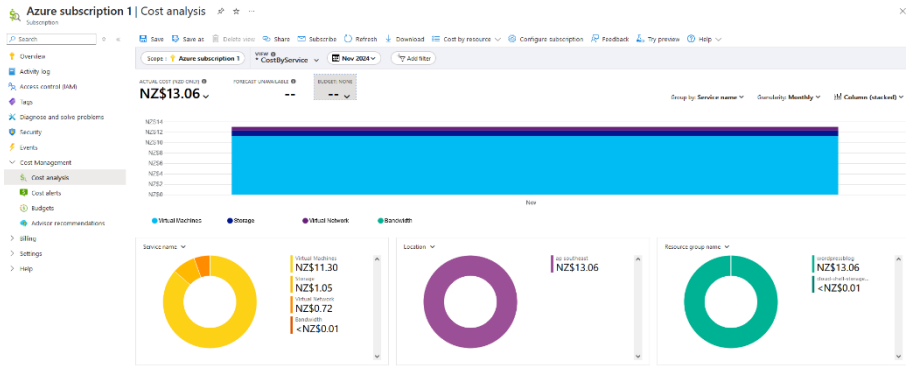


Figure 10: Cost analysis dashboard for the Azure Subscription, utilizing the 'CostByService' built-in view

this dashboard allows me to analyse different aspects of my resources using built-in views, such as CostByService, CostByResource, AccumulatedCosts or DailyCosts. This dashboard can assist in managing costs and seeing what resources the costs are coming from.

Azure also provides an 'Activity log' feature, that allows authorized users to view the activity of certain resources, shown in *Figure 15*. This dashboard shows all activities that have been performed on this resource (the subscription), who initiated the activity, what time, and what the result was.

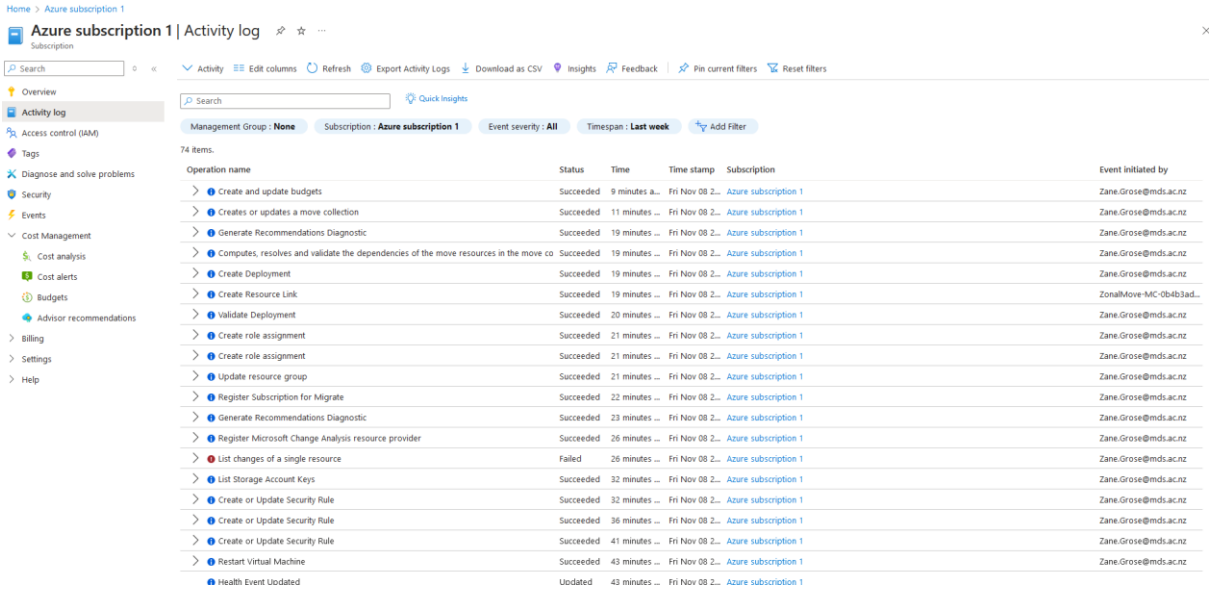


Figure 11: Activity log dashboard for my azure subscription

Reflection

Only using the command line for VM directory management became cumbersome, which encouraged me to allow remote desktop connection to the VM to utilize a desktop. Using a tutorial by Microsoft (Mcinnes, et al., 2024), I installed a xfce4 desktop environment on my VM, then enabled remote desktop connections to the VM by installing xrdp and opening the relevant port in my cloud environment, see *Figure 16*. SSH & RDP ports (22 & 3389 respectively) are restricted to my local IP, this is to only allow connections through these ports from my desktop. HTTP was opened for all connections, this is so the WordPress blog is available to the public and markable by MDS educators. After opening the 'Remote Desktop Connection' application on my Windows local machine,

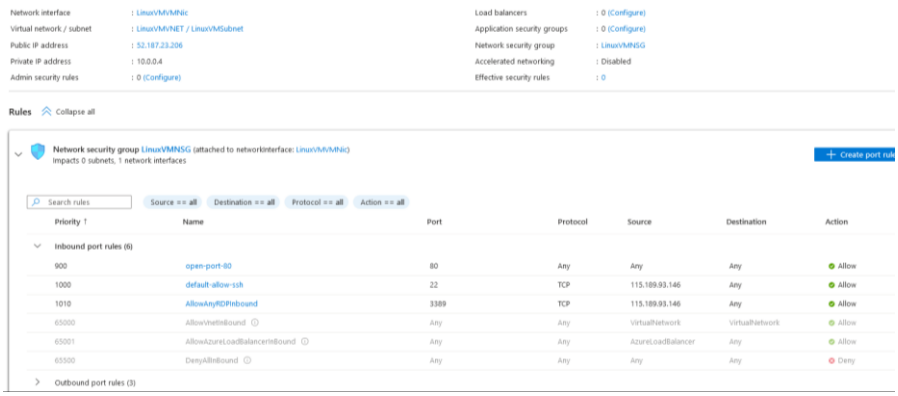


Figure 12: Port settings for the VM, opening SSH, RDP and HTTP ports

I entered my VM's public IP, then my VM user account name and password. This enabled me to access the xfce4 desktop environment through xrdp and find a familiar interface to manage my VM from, see *Figure 17*.

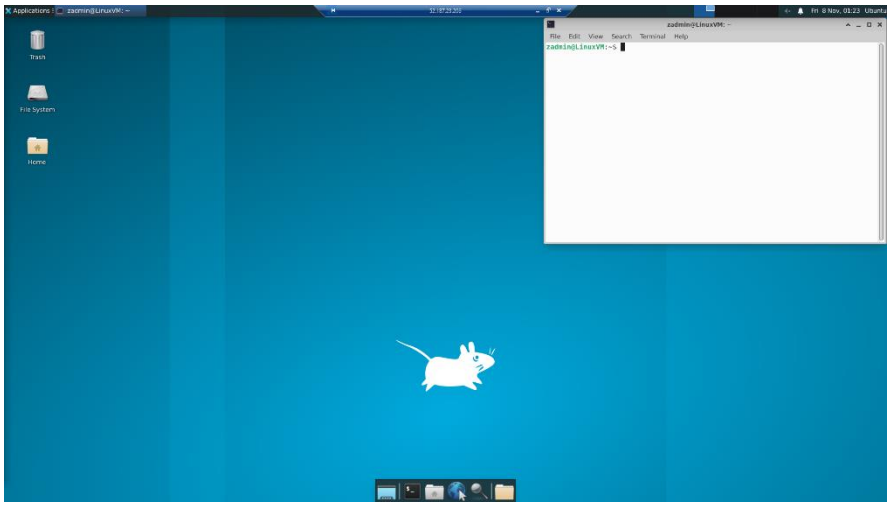


Figure 13: xfce4 desktop environment, accessed through xrdp connection screen through Windows remote desktop connection application

I also found that I was unable to update plugins or upload content from WordPress. To solve this, I added a line of PHP code to the 'config-default.php' file, this line '*define('FS_METHOD', 'direct')*' allows me to use direct file system access, bypassing FTP. This also required me to allow apache to own the wp-content folder on the VM, to enable content updates and uploads. Ensuring that security was considered, I only allowed apache (www-data) to own and read the wp-content folder in the wordpress installation folder, with root owning every other folder/file. This was done utilizing the linux command '*sudo chown -R www-data:www-data /var/www/html/wordpress/wp-content*'.

Acessing the public ip directly directed users to the apache help page, instead of the WordPress blog. To solve this, I copied the 'index.php' file from the wordpress installation folder, changed the 'require __DIR__' (see *Figure 18*) line to direct to the wp-blog-header.php in the wordpress installation folder. This allows access to the wordpress blog by directly entering the public ip, instead of having to write publicIP/wordpress.

```
zadmin@LinuxVM:/var/www/html$ sudo cat index.php
<?php
/**
 * Front to the WordPress application. This file doesn't do anything, but loads
 * wp-blog-header.php which does and tells WordPress to load the theme.
 *
 * @package WordPress
 */

/**
 * Tells WordPress to load the WordPress theme and output it.
 *
 * @var bool
 */
define( 'WP_USE_THEMES', true );

/** Loads the WordPress Environment and Template */
require __DIR__ . '/wordpress/wp-blog-header.php';
```

Figure 14: modified index.php file, available in /var/www/html instead of /var/www/html/wordpress

It would be prudent to enable labs through the Azure interface to directly show and help users (like myself) through the setup process, as the cloud dashboard interface can be quite intimidating for new users. Azure also allows the use of Copilot (AI large language model) to assist with any cloud issues they may encounter, pulling information from their databases like forum discussions, questions, and help articles, which also helped when I could not find a solution using search engines.

Comparison

The cloud hosting option for websites is streamlined, it allows easier scalability, management of costs, quick deployment, and a plethora of activatable features. Compared to traditional hosting, there is less management of hardware, lower initial costs (some servers can cost upwards of $40,000!), and robust redundancy. On-premises solutions require a lot of professional services to setup a robust system, with the need to setup hardware, software, wiring, network, security, and monitoring tools.

Azure provides a lot of features built-in, some of these features consist of load-balancing technology, secure key vaults, virtual machines, DNS names, backup servers, monitoring tools, security tools, and much more. Traditional deployments would require getting subscriptions for each software, configuration on individual devices, and maintaining them. The on-premises solution can become a lot of work for the network administrator.

Conclusion

In conclusion, I successfully deployed a WordPress blog on a LAMP stack using Microsoft Azure. I chose Azure for its intuitive interface, free-tier options, and the convenience of its CloudShell feature. Through CloudShell, I created a Linux VM, installed Apache and MySQL, and used PHP to connect the MySQL database to the WordPress blog. I configured remote desktop access to the VM, gave Apache access to the ‘wp-content’ folder allowing plugin updates, and redirected users from the default Apache page to the WordPress site. Additionally, I explored Azure’s cost management tools and discussed the advantages of cloud deployment over traditional on-premises hosting.

References

Mcinnes, M., kesadae11, ChaseCrum, dbradish, pritamso, liljack17, . . . squillace. (2024, 208 27). *Install and configure xrdp to use Remote Desktop with Ubuntu*. Retrieved from Microsoft Learn: https://learn.microsoft.com/en-us/azure/virtual-machines/linux/use-remote-desktop?tabs=azure-cli

sdwheeler. (2024, 01 23). *Get started with Azure Cloud Shell*. Retrieved from Microsoft Learn: https://learn.microsoft.com/en-us/azure/cloud-shell/get-started/classic?tabs=azurecli

Shim, J., ChaseCrum, eric-mst, cynthn, dbradish, mimckitt, . . . jpconnock. (2024, 08 24). *Install a LAMP stack on an Azure Linux VM*. Retrieved from Microsoft Learn: https://learn.microsoft.com/en-us/azure/virtual-machines/linux/tutorial-lamp-stack